

AI-based Village Pond Planning System

Phase 3 — Interactive Web Application for Pond Siting,
Catchment Analysis and Water-Volume Estimation

Rishi Kharya (Roll No. 12341790)

September 27, 2026

GitHub repository: <https://github.com/krish1809/village-pond>
Live front-end / API: <http://10.1.75.79:5228/> (API docs: [/docs](#))
Demo video: [\[YouTube link\]](#)

Abstract

This phase delivers a fully working web application that lets a user select any land area on an interactive map and, for that area, automatically suggests where a pond could be built, delineates the catchment that would drain into it, and estimates the volume of water it could collect. The backend fetches a real elevation model for the selected block from OpenTopography, reconstructs the terrain, routes surface water across it, and reasons about the result the way a person reading a topographic map would: water runs downhill, gathers in low spots, and a pond only makes sense where water actually collects without sitting inside a flowing watercourse. Rainfall is obtained from the Open-Meteo historical archive and combined with the catchment and the basin geometry to estimate collectable water and the depth required to hold a year's runoff. All results — suggested pond, catchment, contours and the existing drainage network — are overlaid on the map. Nothing is hard-coded to any one location; the same logic runs on any region the user selects.

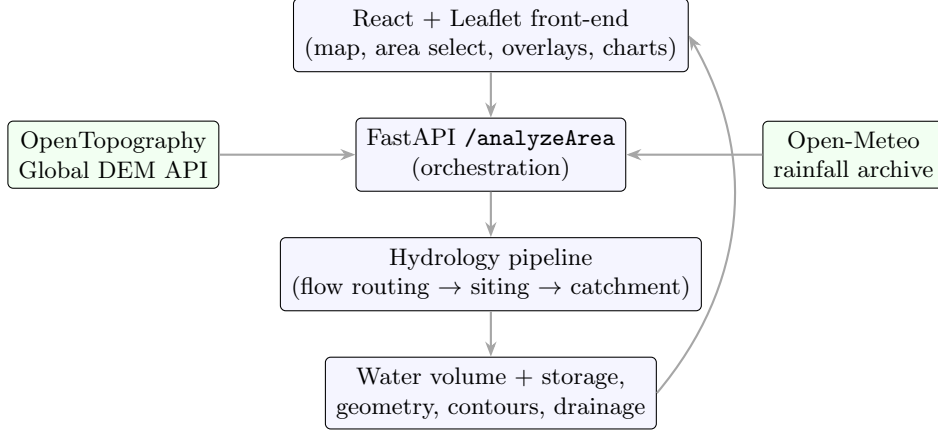
1 Introduction

Choosing a good pond site in a village normally combines three things judged by eye: the shape of the land, how much land funnels runoff toward a point, and how much rain falls there. Doing this manually is slow and gives no quantitative basis for sizing the pond. This phase automates it end to end behind a single web page.

The system meets the phase requirements directly: (i) a fully working front-end; (ii) an option to select the land area on a map; (iii) generation of results for the selected area; (iv) results that include the **suggested pond location**, the **catchment area** and the **expected water volume**; (v) all three overlaid and visualised on the map; and (vi) a solution that is fast and functional within the memory- and storage-limited systems provided. Phase 2's contour-upload route (`POST /analyzeContour`) is retained unchanged; Phase 3 is additive.

2 System Overview and Architecture

The whole system runs as a *single* FastAPI process: it serves both the compiled React front-end (as static files) and the analysis API on one port. This keeps the footprint minimal on the provided containers. The analysis reuses the Phase 2 terrain/hydrology modules unchanged — the only difference is where the elevation model comes from.



Module responsibilities (each a single, non-overlapping concern): `elevation_api` fetches and parses the DEM; `terrain_grid` builds the grid and slope; `catchment` does depression filling, flow direction, flow accumulation and watershed delineation; `pond_locator` scores and ranks candidate sites; `geometry_utils` measures areas/perimeters in metres; `rainfall_api` fetches rainfall; `water_volume` computes runoff, capacity and the storage curve; `contour_lines` generates contours for display; `visualization` renders the PNG figure. The route (`analyze_area`) only wires these together.

3 Methodology

3.1 Elevation model acquisition

For the selected bounding box, the backend requests a Digital Elevation Model (DEM) from the **OpenTopography Global DEM API** (SRTMGL1, 30 m). A single request returns the whole raster for the block. The ASCII-grid (AAIGrid) output format is used so it can be parsed with NumPy without any native GDAL/rasterio dependency. The grid is oriented, cleaned of no-data values, and, if larger than a cap, stride-downsampled to keep computation within the container’s memory.

3.2 Terrain model and slope

The DEM is a regular grid of elevations. Local slope at each cell is computed with the Horn (1981) finite-difference gradient:

$$\text{Slope} = \arctan \left(\sqrt{\left(\frac{dz}{dx}\right)^2 + \left(\frac{dz}{dy}\right)^2} \right). \quad (1)$$

Cell spacing is converted from degrees to metres before slope is interpreted, since coordinates are longitude/latitude.

3.3 Flow routing

Small artificial pits are removed first using the **Priority-Flood** algorithm (Barnes et al., 2014), an $O(n \log n)$ depression fill that processes cells outward from the boundary in elevation order. **D8 flow direction** (O’Callaghan & Mark, 1984) then assigns each cell to its steepest-descent

neighbour, where the downhill slope toward a neighbour is

$$S_d = \frac{z_{\text{current}} - z_{\text{neighbour}}}{\text{distance}}. \quad (2)$$

Flow accumulation — the number of upstream cells draining through each cell — is then computed by processing the flow network in topological order. High accumulation marks where surface water concentrates.

3.4 Choosing where a pond makes sense

Flow accumulation alone is not enough: a channel carries much water but is not a place to build a pond, while a dry knoll is not either. Two terrain measures are combined. The **Topographic Wetness Index** captures the tendency of water to gather while accounting for slope,

$$\text{TWI} = \ln \left(\frac{\text{upslope area}}{\tan(\text{slope})} \right), \quad (3)$$

and the **Topographic Position Index** captures whether a cell sits lower than its surroundings,

$$\text{TPI} = z_{\text{cell}} - \bar{z}_{\text{neighbourhood}}. \quad (4)$$

Both are normalised to $[0, 1]$; since a *lower* position is favourable, its sign is flipped, giving the suitability score

$$\text{Score} = 0.5 \text{TWI}_{\text{norm}} + 0.5 (-\text{TPI})_{\text{norm}}. \quad (5)$$

Three hard feasibility filters are then applied: cells steeper than 15° are excluded (earthen ponds are not built on steep ground); cells belonging to the existing **drainage/channel network** (flow accumulation above the 97th percentile, plus a small buffer) are excluded, because a pond must be fed *by* a watercourse, not sit *inside* it; and a candidate whose flood-fill footprint cannot be bounded within a growth cap is rejected, since that indicates broad floodplain-like ground rather than a containable basin. Candidates are then chosen greedily with a minimum-separation radius so the top sites are genuinely distinct regions.

3.5 Catchment delineation

For each selected site the **catchment** (watershed) is traced by walking the D8 flow network *backwards* from the site: a neighbouring cell is included whenever its flow direction points toward the current cell, continuing until no more upstream contributors are found. On flat terrain the resulting watershed can be several disconnected pieces; all pieces are retained.

3.6 Metric geometry

The cell masks are converted into polygons by unioning the exact cells (not an approximating hull, which was found to overestimate area). Because longitude/latitude are unsuitable for measuring area, polygons are projected into the appropriate **UTM zone** — selected automatically from the block’s location — so area and perimeter are computed in metres. Boundaries drawn on the map are additionally smoothed for display only; the reported areas are always measured on the exact polygons.

3.7 Rainfall

Mean annual rainfall is obtained from the **Open-Meteo historical archive**, averaged over the last several complete years of daily precipitation for the block’s centre. Every call has a timeout and, if the service is unreachable, the system falls back to a documented default annual rainfall so a volume is always produced; the response marks which source was used.

3.8 Water volume and storage

Two independent limits are computed. The annual runoff the catchment delivers uses a Rational-method estimate,

$$V_{\text{runoff}} = C \cdot P \cdot A_{\text{catchment}} \quad (\text{m}^3/\text{yr}), \quad (6)$$

where P is mean annual rainfall (m/yr), $A_{\text{catchment}}$ is the catchment area (m²) and C is a runoff coefficient (default 0.3, adjustable). The basin’s storage capacity is measured directly from the DEM over the pond footprint,

$$V_{\text{capacity}} = \sum_{\text{cells}} (w - z_i) \cdot A_{\text{cell}}, \quad (7)$$

where w is the water-surface level. The realistically collectable volume is bounded by both — the catchment cannot deliver more, and the basin cannot hold more:

$$V_{\text{collectable}} = \min(V_{\text{runoff}}, V_{\text{capacity}}). \quad (8)$$

Finally, a stage–area–volume (**storage curve**) is built by flooding the basin at increasing depths (bounded to the catchment), and the **required depth** — the depth at which the stored volume reaches one year’s runoff — is reported.

4 Front-end

The front-end is a React + Leaflet single-page application. The user can (i) search a village/-town/city by name (OpenStreetMap geocoding) and fly the map there; (ii) select the land area either by “Use current map view” or by clicking two corners of a rectangle; and (iii) run the analysis. Results are drawn on the map: the DEM’s elevation **contours**, the existing **drainage network** (blue), each ranked **pond footprint** (filled region, labelled with its area), its **catchment** (dashed outline) and a marker at the suggested **pond location**. A side panel shows, per site, the catchment area, pond footprint, annual runoff, basin capacity, collectable volume, required depth and a storage-curve chart, plus a monthly rainfall chart for the area. Results can be downloaded as a rendered **PNG**, as **GeoJSON** (ponds, catchments and drainage, for GIS tools) and as the full **JSON** response. A plain OpenStreetMap base map is used, with Esri satellite imagery available as a toggle.

5 API Documentation

Endpoint. POST /analyzeArea

Request body (JSON).

```
{ "bbox": [min_lon, min_lat, max_lon, max_lat],
  "num_candidates": 3,           // 1-10, optional (default 3)
  "runoff_coefficient": 0.3 }    // optional (default 0.3)
```

A query parameter `format=image` returns a rendered PNG of the same result instead of JSON.

Response (`format=json`). An object containing:

- **candidates** — ranked list; each has `location` (lat, lon, elevation, slope), `pond_footprint` (area, perimeter, GeoJSON boundary, assumed depth), `catchment` (area, perimeter, GeoJSON boundary), `suitability` (the scoring factors), `water_volume` (annual runoff, storage capacity, collectable volume, limiting factor) and `storage` (required depth, stage–area–volume curve).
- **contours** — elevation contour lines generated from the DEM.
- **drainage** — GeoJSON of the existing stream/channel network.
- **rainfall** — mean annual rainfall, source and monthly climatology.
- **area_summary** — bbox, grid resolution, elevation range, DEM source.
- **metadata** — a step-by-step processing log and total time.

Error responses. 422 — the selected area is malformed, too large or too small, or has no feasible pond site; 502 — the DEM service could not be reached; 500 — an unexpected processing error (with the failing stage). Full interactive documentation is auto-generated at </docs>.

6 Results and Demonstration

Figure 1 shows the system run on the provided sample region (near Raipur, Chhattisgarh), the same terrain used in Phase 2. The 30 m DEM (elevation 266–298 m) reproduces the known landscape: the diagonal blue band is the main stream valley, with tributary channels branching from it. All three suggested ponds are placed on drainable ground *away* from the blue drainage network, confirming the channel-exclusion logic. Table 1 lists the numbers.

Table 1: Ranked pond candidates for the sample region (grid 65×101 , runoff coefficient $C = 0.3$). Catchment area is always $\geq$ pond footprint, as expected.

Rank	Location (lat, lon)	Catchment (m ²)	Pond (m ²)	Annual runoff (m ³)	Collectable (m ³ /yr)	Req. depth (m)
1	21.2592, 81.2986	6 202	2 658	2 047	1 901	2.5
2	21.2556, 81.2964	2 658	1 772	877	877	1.0
3	21.2594, 81.2925	2 658	1 772	877	877	1.0

The relationship between the quantities is internally consistent: the catchment (water source) is always at least as large as the pond footprint (where water collects), and the collectable volume is the smaller of what the catchment delivers and what the basin can hold. At this DEM resolution (30 m) over gently sloping ground the catchments are modest; on terrain with stronger relief the same pipeline produces correspondingly larger catchments and volumes. The end-to-end analysis, including the live DEM fetch, completes in roughly 12 seconds and is cached thereafter.

7 Deployment, Performance and Scaling

The application is deployed on the provided university server and reachable at <http://10.1.75.79:5228/>. The React app is built and served as static files by the same FastAPI process, so the entire system is one process behind one URL, started detached (`nohup`) on the container’s internal port 5000 (forwarded to the external 5228). Dependencies are installed into the pre-

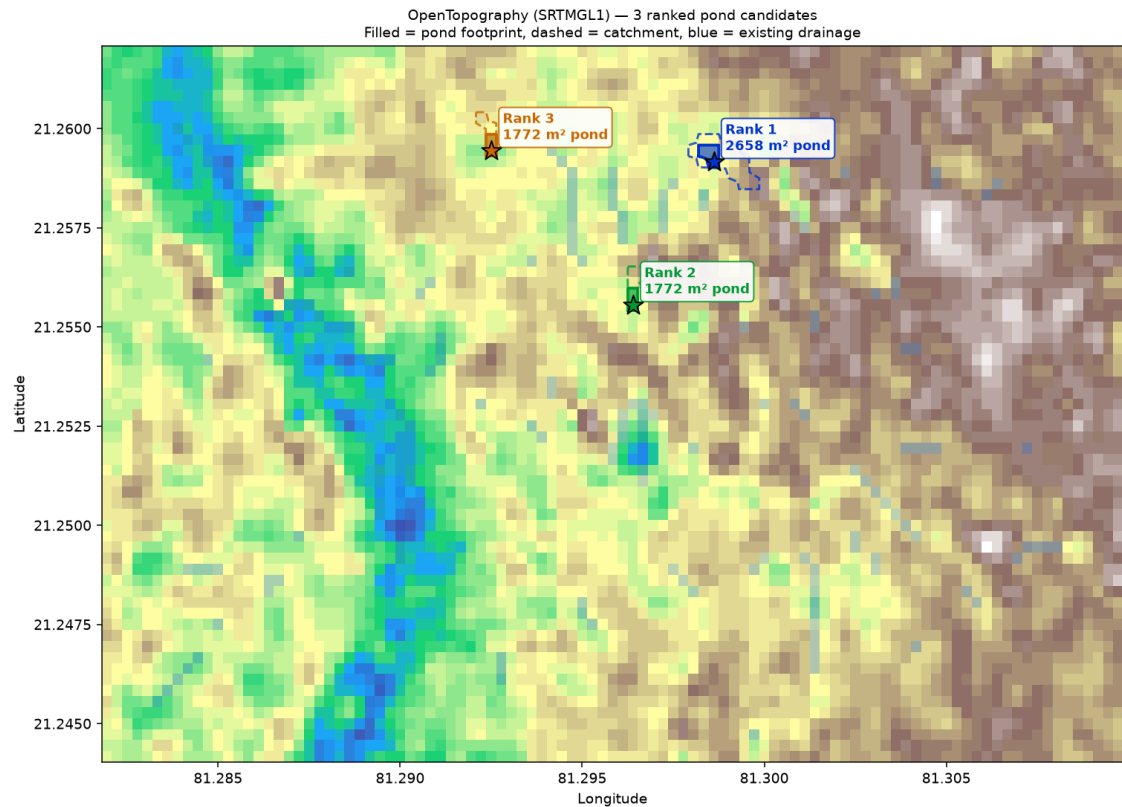


Figure 1: Analysis of the sample region. Terrain shading is the DEM; blue is the existing drainage network; filled shapes are the suggested pond footprints and dashed outlines their catchments. The ponds are correctly kept off the watercourses.

existing environment rather than a duplicate virtual environment, to respect the container’s small storage.

The provided containers are constrained (memory limit 512 MB). The design stays within this: the server’s resident memory is about 160 MB; matplotlib is imported lazily so it is only loaded when a PNG is actually requested; the analysis grid is capped and downsampled when necessary; and each DEM fetch and rainfall lookup is cached in memory (keyed by area), so re-analysing the same or an adjacent block is instant and makes no repeat external calls. Oversized area selections are rejected up front with a clear message rather than being allowed to fan out into an expensive request. The single-process, dependency-light design means the same build can run on any of the four provided systems without modification.

The backend is covered by 71 automated tests (parser, terrain grid, flow/catchment, pond locator, geometry, elevation parsing, rainfall, water volume and the image format), all passing.

8 Use of AI Tools

In line with the course’s LLM-usage policy, Claude (Anthropic) was used as a coding and debugging assistant during development: to draft initial implementations of individual modules, to explain the underlying terrain-analysis algorithms, and to help diagnose real issues found during testing — for example, correcting a case where the pond footprint could exceed its catchment (resolved by clipping the footprint to the catchment), and improving the drainage visualisation. All technical content and implementation decisions were reviewed and are understood by the

author, including the formulas and design trade-offs described in Section 3.

9 Conclusion

This phase delivers a working, deployed web application that turns a user-selected map region into concrete, ranked pond recommendations with real catchment areas and water-volume estimates, all visualised on the map. It fetches genuine elevation and rainfall data for the chosen area, applies a transparent and explainable hydrological pipeline, and correctly keeps suggested ponds out of existing watercourses — while running comfortably within the memory and storage limits of the provided systems.